

ASP.NET Core Kestrel Web Server

Back to: [ASP.NET Core Web API Tutorials](#)

Kestrel Web Server in ASP.NET Core

ASP.NET Core is a cross-platform framework that allows us to build high-performance, modern web applications. At the heart of every ASP.NET Core application lies a **web server** that listens for incoming HTTP requests and routes them into the application pipeline. On Windows, we often rely on IIS; however, for **cross-platform compatibility** and **lightweight deployments**, ASP.NET Core provides its own built-in server, called **Kestrel**.

Kestrel is designed to be **fast, efficient, and cross-platform**, capable of handling thousands of concurrent connections. Whether you deploy your application on Windows, Linux, or macOS, Kestrel makes it possible. Before we dive into Out-of-Process Hosting or reverse proxy scenarios, it is essential first to understand the Kestrel web server.

What is Kestrel Web Server?

Kestrel is the **Default Cross-Platform Web Server** built into ASP.NET Core. It is lightweight, high-performance, and capable of handling thousands of concurrent connections. Unlike IIS, which only runs on Windows, Kestrel can run on **Windows, Linux, and macOS**, making it the backbone of ASP.NET Core's cross-platform hosting. In development, you can run apps directly with Kestrel. In production, Kestrel often runs behind a **reverse proxy**, such as IIS, Nginx, or Apache, to provide advanced features like SSL termination, load balancing, and process management.

Key Points:

- Cross-platform: runs on Windows, Linux, and macOS.
- Default server for ASP.NET Core apps.
- Lightweight and high-performance.
- Can serve requests directly (standalone).
- Often paired with reverse proxies in production for security and scaling.

Analogy: Think of Kestrel as the **engine of a car**. You can drive a car directly with just the engine, but in a big city (production), you usually have **traffic controllers and road systems (IIS, Nginx)** to organize and manage things more efficiently.

What is the Process Name in the Case of the Kestrel Web Server?

When running on Kestrel, your application does not run inside the IIS worker process (w3wp.exe) or IIS Express (iisexpress.exe). Instead, the process name depends on your deployment type. If you run the app as a **Framework-Dependent Deployment**, you'll see dotnet as the process name. If you publish as a **Self-Contained Deployment**, the process name will be the name of your application's executable (e.g., MyApp.exe).

Key Points:

- No w3wp.exe or iisexpress.exe.
- Framework-Dependent → process name = dotnet.
- Self-Contained → process name = application exe (e.g., MyApp.exe).

Analogy: Imagine two shops: one is a **Rented Space Inside a Mall (Framework-Dependent Deployment)**, where you run under .NET), and the other is a **Standalone Shop with Its Own Board (Self-Contained Deployment)**, with your own executable name). Both serve customers, but their "nameplate" looks different.

What is Self-Contained Deployment in ASP.NET Core?

A **Self-Contained Deployment (SCD)** bundles your app with the .NET runtime, libraries, and all dependencies. This means the target machine does not need to have .NET Runtime installed; the app carries everything it needs. The tradeoff is that the deployment package is larger. In this mode, the process name will be your application's exe (e.g., MyApp.exe). This is commonly used in scenarios where you can't control the environment and can't guarantee that .NET will be preinstalled on the server.

In **Self-Contained Deployment**, the published output includes a **Platform-Specific Executable** — on Windows, it is MyApp.exe, and on Linux/macOS, it is simply MyApp (without an extension).

Key Points:

- The app includes the .NET runtime and all dependencies.
- The target machine doesn't need .NET installed.
- Larger publish size.
- Process name = your app's exe.
- Best for environments with no preinstalled runtime.
- Useful for containers, cloud environments, or systems without the .NET runtime installed.

Analogy: Think of it like **packing your own lunchbox with food and utensils**. You don't care what's available at the picnic spot because you have carried everything with you.

What is Framework-Dependent Deployment in ASP.NET Core?

A **Framework-Dependent Deployment (FDD)** assumes the target machine already has the correct .NET runtime installed. The published app only contains your application code and required libraries. This makes the package smaller but requires the server to have .NET available. In this mode, the process name is always 'dotnet', since you run the app using 'dotnet MyApp.dll'.

In **Framework-Dependent Deployment**, the published output contains your application code as a **DLL file (MyApp.dll)**. This DLL requires the .NET runtime to be already installed on the target machine.

Key Points:

- Relies on the .NET runtime being installed on the target machine.
- Smaller publish size.
- Process name = dotnet.
- Requires runtime compatibility on the host.
- Suitable when multiple apps share the same runtime.

Analogy:

This is like **renting a kitchen** in a shared restaurant. You bring your own recipe and staff, but rely on the restaurant's ovens, stoves, and utilities to cook food. If they don't have the right tools, you can't cook.

What is Publish in ASP.NET Core?

When you build an ASP.NET Core application in Visual Studio, it creates intermediate build files (DLLs, config files, etc.) in the bin and obj folders. These files are fine for **development** but **not ready for deployment**.

Publishing means preparing your application so it can run on a target environment (server, cloud, Docker, etc.) by:

- Collecting all **compiled assemblies** (your app DLLs + referenced DLLs).
- Including all **static files** (CSS, JS, images, Razor views).
- Copying **configuration files** (appsettings.json, logging, etc.).
- Optionally including the **.NET runtime** (for Self-Contained deployments).
- Producing a **final folder or package** that you can deploy anywhere.

So, **Publish = Final Packaged Output** that the server can execute.

Why is Publishing Needed?

- Development builds (bin/Debug) may include unnecessary files.
- Published builds are optimized for **deployment and runtime execution**.
- Helps you target **different environments** (Windows, Linux, macOS).
- Allows you to choose the **deployment mode**:

- **Framework-Dependent Deployment (FDD)** → requires .NET runtime installed on server.
- **Self-Contained Deployment (SCD)** → carries runtime with app.

Analogy: Think of publishing like **packing your luggage for a trip**. During development, everything is scattered in your room (bin/debug). When publishing, you pack only the necessary clothes, shoes, and items into a suitcase (publish folder), ready for the journey (deployment).

How to Publish Code in Visual Studio (Step by Step)

Step 1: Right-Click Project

- In **Solution Explorer**, right-click on your project (not the solution).
- Select **Publish...** from the menu.

Step 2: Choose Publish Target

Visual Studio shows different publish options:

- **Folder** → Outputs files to a folder (for manual deployment).
- **IIS, FTP, or Azure** → Deploys directly to IIS, remote server, or cloud.
- **Docker** → Publishes to a container image.

For testing Kestrel or understanding deployment, start with the **Folder**.

Step 3: Configure Profile

- If you choose **Folder**, Visual Studio asks where to put the published files (e.g., bin\Release\net8.0\publish).
- Click **Finish**.

Step 4: Adjust Deployment Settings (Optional)

Open the publish profile settings:

- **Configuration** → Debug or Release (use Release for production).
- **Deployment Mode** →
 - Framework-Dependent (default, smaller size).
 - Self-Contained (includes runtime, bigger size).
- **Target Runtime** → Choose OS/CPU (e.g., win-x64, linux-x64).

Step 5: Publish

- Click the **Publish** button.

- Visual Studio compiles your app, copies required files, and creates a final **publish folder**.

Running from Visual Studio with HTTP/HTTPS Profiles

When we hit **F5** (Debug) or **Ctrl+F5** (Run without Debug) in Visual Studio using **HTTP** or **HTTPS** profiles:

- Visual Studio **does not publish** the app.
- It simply **builds** our project into the `bin\Debug\netX.Y\` (or `bin\Release`) folder.
- Then it launches the app using the **.NET Runtime**.

That means your app is running as a **Framework-Dependent Deployment (FDD)** by default.

Why FDD?

- The output is **not self-contained**; it depends on the shared .NET runtime installed on your machine.
- The **process name** you will see in the Task Manager is 'dotnet' (not your app's EXE).

Example to Understand Kestrel Web Server in ASP.NET Core

Consider a simple API controller that shows hosting details when using Kestrel. When you run it as FDD, the process name is dotnet. When you run it as SCD, the process name is the same as your app's executable name. This demonstrates how Kestrel adapts based on deployment type but always remains the web server handling requests.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Hosting.Server;
using System.Diagnostics;

namespace MyFirstWebAPIProject.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class TestController : ControllerBase
    {
        // The IServer interface represents the actual server implementation
        // (IISHttpServer or KestrelServer)
        private readonly IServer _server;

        // Constructor Dependency Injection - ASP.NET Core injects the active
        // IServer service
        public TestController(IServer server)
        {
            _server = server;
        }
    }
}
```

```

// GET: api/test/ping
[HttpGet("ping")]
public IActionResult Ping()
{
    // Get the current process information (e.g., w3wp, iisexpress,
dotnet)

    var proc = Process.GetCurrentProcess();

    // Return hosting environment details as JSON response
    return Ok(new
    {
        message = "Hosting Info",           // Custom message
        pid = Environment.ProcessId,        // Numeric process ID
        process = proc.ProcessName,         // Running process name
        serverType = _server.GetType().Name, // Hosting server type
        (IISHttpServer or KestrelServer)
        machine = Environment.MachineName   // Server machine name
    });
}
}
}

```

Running the Application using Kestrel Server:

To use the Kestrel Web Server to host and run our application in Visual Studio, we need to run the application using either **HTTP or HTTPS Profiles**. Let us run the application using the **HTTPS** profile. Once you run the application using the HTTPS profile, you need to observe two things.

First Thing:

First, it will launch the command prompt and host the application using the Kestrel Web Server, as shown in the image below. Here, you need to focus on the URL and Port Number, which should match the exact URL and port number specified in the **HTTPS** profile of the launchSettings.json file.

```

info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7191
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5021
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Projects\MyFirstWebAPIProject\MyFirstWebAPIProject

```

Second Thing:

Secondly, it opens the default web browser with the Swagger page, which uses the HTTPS protocol. Now, if you hit the **api/Test/ping** endpoint, then you will see the following output:

Request URL

`https://localhost:7191/api/Test/ping`

Server response

Code	Details
200	Response body <pre>{ "message": "Hosting Info", "pid": 21152, "process": "MyFirstWebAPIProject", "serverType": "KestrelServerImpl", "machine": "LAPTOP-6P5NK25R" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Thu,25 Sep 2025 14:07:38 GMT server: Kestrel</pre>

Why Process Name = Project Name in Visual Studio

When we run an ASP.NET Core Web application directly in Visual Studio (using the **Project profile** → **HTTP/HTTPS**):

1. Visual Studio builds your project into bin/Debug/net8.0/.
2. It generates both:
 - **MyFirstWebAPIProject.dll** (your app assembly).
 - **MyFirstWebAPIProject.exe** (a small framework-dependent executable host).
3. Visual Studio runs the **generated exe**, not dotnet MyFirstWebAPIProject.dll.
4. That's why the **process name matches your project name** (MyFirstWebAPIProject).

Note: This .exe is still **Framework-Dependent**, as it simply boots up and calls the installed .NET runtime on your machine. It is not a Self-Contained Deployment.

The **Kestrel Web Server** is the foundation of ASP.NET Core hosting. It provides a fast, lightweight, and cross-platform server that works across Windows, Linux, and macOS. Whether you run it **standalone** or behind a **reverse proxy**, understanding Kestrel is essential for mastering ASP.NET Core hosting models. By understanding its process names, deployment styles (Self-Contained vs. Framework-Dependent), and how to run applications on it, you are prepared to transition to the **Out-of-Process hosting model**, where Kestrel works in conjunction with IIS, Nginx, or Apache to deliver enterprise-grade applications.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information